SYSTEM SPECIFICATION

RF PRESTIGE

E-commerce platform for luxury pre-owned automobiles

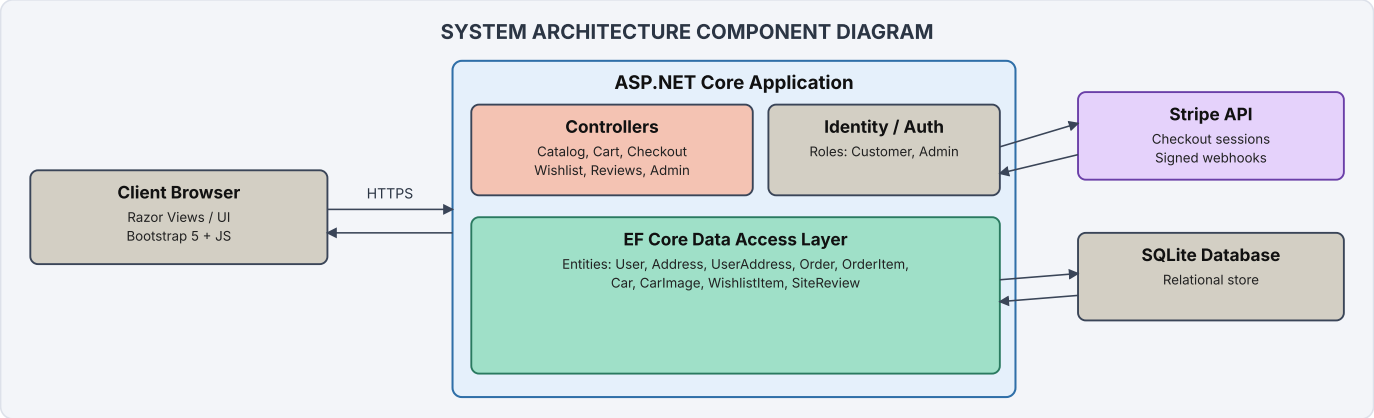
DOCUMENT REF	SPEC-P2-2026	**PROJECT NAME**	RF Prestige
VERSION	2.2 (Schema revision: N:M User–Address)	**ARCHITECTURE**	ASP.NET Core MVC (C#)
AUTHORS	Cătălinoiu L. Roberto Țicleanu Ș. Alexandru	**TARGET MARKET**	Premium Automotive (RO)

1. Executive Summary & Niche Definition

RF Prestige is an e-commerce platform engineered specifically for the online acquisition of verified pre-owned luxury and high-performance automobiles. The target demographic comprises business owners and senior professionals in Romania aged 30–55 with acquisition budgets spanning €60,000 to €250,000. Every vehicle is identified by its unique Vehicle Identification Number (VIN).

2. System Architecture & API Endpoint Table

The system utilizes ASP.NET Core MVC with Entity Framework Core and SQLite. Below is the component breakdown and the complete list of system API endpoints.



Complete API Routes & Access Levels

Method	Route Path	Description	Access Level
GET	`/Catalog`	Retrieves full list of vehicles with basic search (Make, Model, VIN).	Public
GET	`/Catalog/Details/{vin}`	Vehicle details and image gallery, looked up by VIN.	Public
POST	`/Cart/Add`	Adds a single vehicle (by VIN) to the session cart.	Public
GET	`/Checkout`	Initiates multi-step checkout session (delivery address and option).	Public / User
POST	`/Checkout/CreateSession`	Saves or links the shipping address, creates the Pending order and the Stripe Checkout session.	Authenticated
POST	`/Webhooks/Stripe`	Handles Stripe webhook event callbacks.	Stripe System
GET	`/Orders/History`	Fetches past order history for the logged-in user.	Authenticated
GET/POST	`/Account/Addresses`	Lists, links, replaces and unlinks the user's saved addresses (UserAddress).	Authenticated
GET	`/Wishlist`	Shows the user's wishlist.	Authenticated
POST	`/Wishlist/Toggle`	AJAX add / remove of a vehicle (by VIN) in the wishlist.	Authenticated
GET	`/Reviews`	Lists the site reviews with the average rating.	Public
POST	`/Reviews/Create`	Submits a site review (rating and comment).	Authenticated
GET/POST	`/Admin/Cars/*`	CRUD operations for luxury car inventory.	Admin Only
GET/POST	`/Admin/Orders/*`	Order status management and revenue logs.	Admin Only
GET/POST	`/Admin/Reviews/*`	Review moderation (list and delete).	Admin Only

3. Requirements A1–A12 & Chosen Bonus Features

The following table outlines how each requirement will be implemented in 1–2 concise sentences along with assigned module owners.

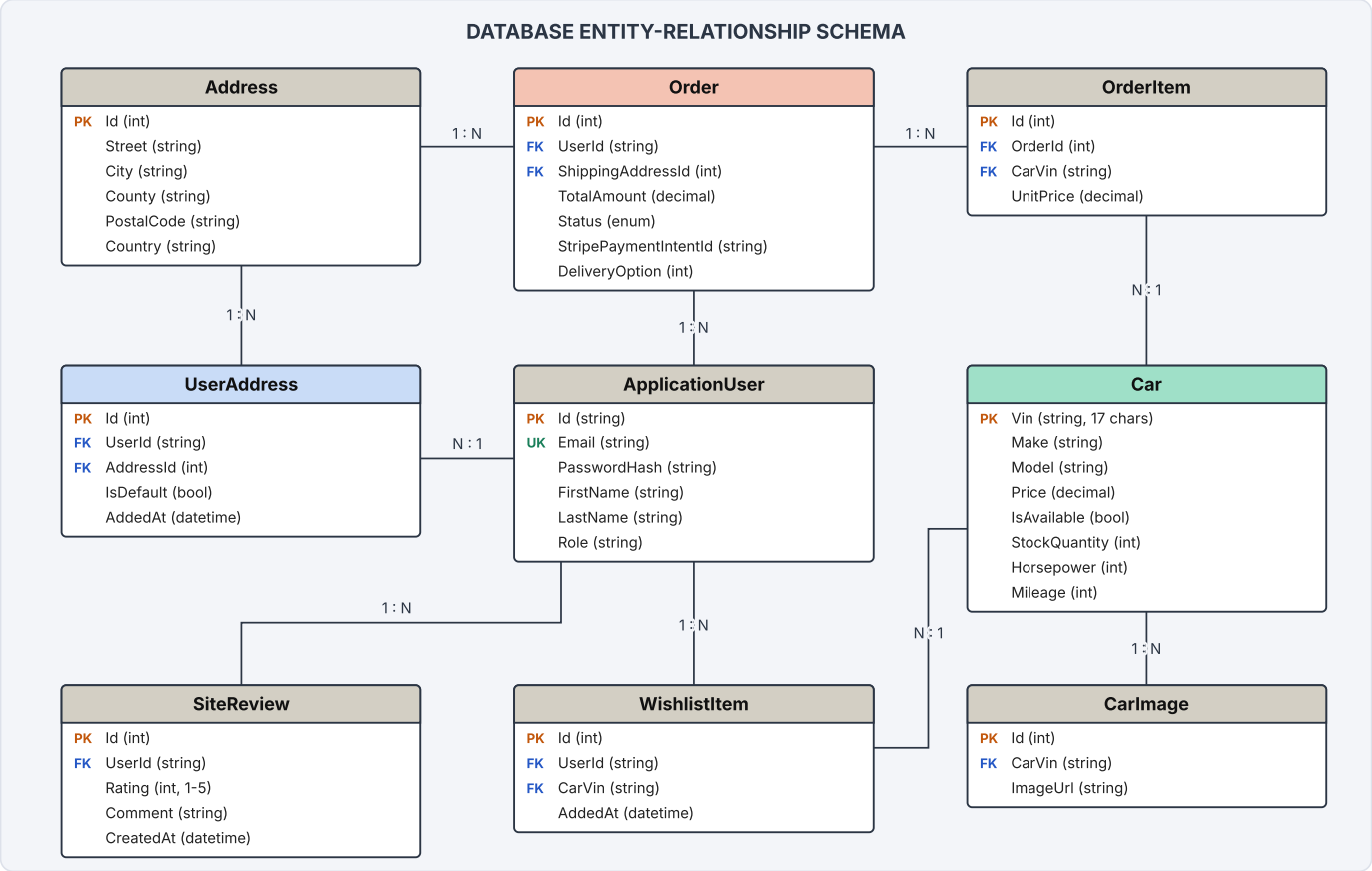
Code	Requirement	Implementation Summary (1–2 Sentences)	Owner
A1	**Product Catalog**	Implemented via Razor views rendering a responsive grid of available luxury cars filtered by category. Vehicles are retrieved with EF Core optimized queries containing their `CarImage` paths and key specs.	Alexandru
A2	**Basic Search**	Implemented as a header search bar executing LINQ queries against vehicle `Make`, `Model`, and `Vin` fields. Results update dynamically on the catalog view page.	Alexandru
A3	**Shopping Cart**	Implemented using session-backed memory stores for anonymous users and EF Core persistent cart items for logged-in accounts, with each cart line referencing a car by VIN. Computes delivery selection (€490 vs €0) and totals.	Roberto
A4	**User Accounts**	Implemented using ASP.NET Core Identity with BCrypt password hashing. Manages registration, authentication cookies, role separation (`Customer` vs `Admin`) and saved delivery addresses, kept in `Address` and linked to users many-to-many through `UserAddress`.	Roberto
A5	**Checkout**	Implemented as a multi-step form capturing delivery preferences, a structured address (street, city, county, postal code, country) and terms acceptance. The address is stored once in `Address` (an identical existing row is reused), linked to the user through `UserAddress` when “Save this address” is ticked, and a Pending order referencing it is generated in EF Core prior to payment.	Roberto
A6	**Sandbox Payment**	Implemented via Stripe Checkout API integration in test mode. A dedicated controller receives signed webhooks from Stripe to confirm order payment asynchronously.	Roberto
A7	**Stock Management**	Implemented by setting `IsAvailable = false` and decrementing stock on the car (identified by VIN) upon payment confirmation. Uses EF Core row locking transactions to prevent duplicate sales.	Roberto
A8	**Order History**	Implemented as an authenticated user dashboard listing past orders, purchased vehicle VINs (via `OrderItem`), status tags, and downloadable invoices.	Alexandru
A9	**Custom Admin Panel**	Implemented as a custom controller/view area (no third-party admin scaffold) allowing admins to manage inventory, update order statuses, moderate site reviews, and view sales metrics.	Alexandru
A10	**Basic Security**	Implemented using native CSRF anti-forgery tokens on all POST forms, EF Core parameterized queries against SQLi, and HTML encoding against XSS attacks.	Roberto
A11	**Responsive Design**	Implemented using Bootstrap 5 flex grid utility classes and custom media queries. Ensures full visual compliance across desktop, tablet, and mobile browsers.	Alexandru
A12	**Automated Tests**	Implemented using xUnit test suite with Moq framework. Covers core business logic including pricing totals, cart state updates, VIN validation, stock concurrency rules and address deduplication / linking.	Alexandru

Chosen Bonus Features

Code	Bonus Feature	Points	Implementation Summary	Owner
B1	**Filtered Search**	5 pts	Sidebar filter controls allowing clients to refine search by price range, manufacture year, engine specs, transmission, and power.	Alexandru
B2	**Wishlist / Favorites**	2 pts	AJAX toggle allowing authenticated users to bookmark cars into a persistent wishlist, stored in the dedicated `WishlistItem` table (user + car VIN) and shown on a wishlist page.	Roberto
B3	**Recommendations**	4 pts	Algorithm on vehicle details page suggesting 3 similar cars based on price proximity (±15%) and engine category.	Roberto
B7	**Reviews & Ratings**	4 pts	1–5 star rating and comment submission by authenticated users about the site, stored in the dedicated `SiteReview` table and subject to admin moderation (admins can delete inappropriate reviews).	Alexandru

4. Data Model Entity–Relationship Diagram

The complete Entity-Relationship diagram illustrates database entities, primary key constraints, foreign key mappings, and cardinalities within SQLite. Cars are identified by their VIN, delivery addresses live in a separate table shared with users through the `UserAddress` join table (many-to-many), and the wishlist and the site reviews are two independent tables.



Relationships

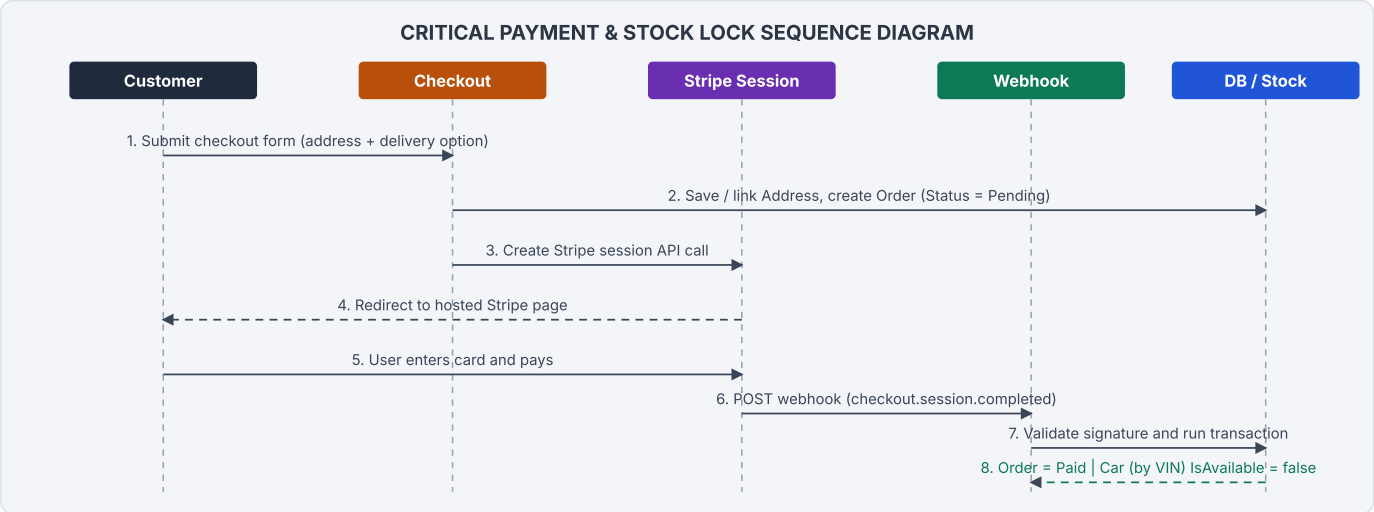
Relationship	Cardinality	Description
ApplicationUser ↔ Address	N : M	A user can save several delivery addresses and an address can be saved by several users. Implemented by `UserAddress` (`UserId` + `AddressId`).
ApplicationUser → UserAddress	1 : N	Each link row belongs to one user and carries that user's `IsDefault` flag.
Address → UserAddress	1 : N	Each link row points to one shared address record.
ApplicationUser → Order	1 : N	A user can place many orders.
Address → Order	1 : N	Each order ships to one address (`ShippingAddressId`).
Order → OrderItem	1 : N	An order contains one or more vehicle lines.
Car → OrderItem	1 : N	An order line references one car by `CarVin`.
Car → CarImage	1 : N	A car has an image gallery.
ApplicationUser ↔ Car	N : N	Implemented by `WishlistItem` (`UserId` + `CarVin`).
ApplicationUser → SiteReview	1 : N	A user writes reviews about the site; reviews are not linked to a car.

Design notes

- `Car.Vin` (17 characters) is the primary key and replaces the former numeric `Id`; every table that references a car stores `CarVin`.
- `UserAddress` has a unique pair (`UserId`, `AddressId`), so an address cannot be linked twice to the same user. `IsDefault` lives here because "default" is per user; at most one default per user (service-layer rule or filtered unique index on `UserId` where `IsDefault = 1`).
- **Address immutability:** `Address` rows are never edited in place, because they are shared and referenced by past orders. "Edit" creates (or reuses) an `Address` and repoints that user's link.
- **Deduplication:** saving an address first looks for an existing row with the same street, city, county, postal code and country and links to it; otherwise a new row is created.
- **Delete** removes only the `UserAddress` link. The `Address` row is kept while any order references it; unlinked, order-free addresses may be purged.
- `WishlistItem` has a unique pair (`UserId`, `CarVin`). `SiteReview.Rating` is restricted to 1–5.
- `Car` also carries a `RowVersion` concurrency token (omitted from the diagram) used by the optimistic locking described in Section 8.

5. Critical Payment Flow Sequence Diagram

The critical payment flow sequence documents step-by-step communication between the User, Checkout Controller, Stripe API Processor, Webhook Listener, and the Order / Stock Inventory Database.



> **Idempotency Assurance:** The webhook controller verifies the Stripe signature secret and checks whether `Order.Status` is already set to `Paid` before applying inventory modifications, preventing duplicate executions.

6. User Interface Wireframes — Catalog, Product Detail, Wishlist & Reviews

The user interface layouts prioritize vehicle imagery, key technical specifications, and clear call-to-action buttons. Every vehicle is shown with its VIN.

6.1 Product Catalog Wireframe

```
RF PRESTIGE | Catalog | Wishlist | Reviews | Cart (1) | Account          Search (Make/Model/VIN): [ Porsche 911... ] [Search]

FILTERS (B1)
Category: [x] Sports
Price: €60k - €250k
Power: [ 500+ HP ]
Fuel: [ Hybrid/EV ]

[ IMAGE: Porsche 911 GT3 ]
2023 Porsche 911 GT3
4.0L Flat-6 | 510 HP | 4,200 km
VIN: WP0ZZZ99ZPS291047
€215,000
[ View Details ] [♥ Wishlist]

[ IMAGE: Ferrari F8 Tributo ]
2022 Ferrari F8 Tributo
3.9L V8 Turbo | 720 HP | 1,800 km
VIN: ZFF92LLA9N0267841
€248,000
[ View Details ] [♥ Wishlist]
```

6.2 Product Page Wireframe

```
CATALOG > SPORTS CARS > 2023 PORSCHE 911 GT3   (/Catalog/Details/WP0ZZZ99ZPS291047)

[ MAIN HIGH-RES VEHICLE IMAGE ]
[Thumb 1] [Thumb 2] [Thumb 3] [Thumb 4]
(CarImage gallery)

2023 Porsche 911 GT3   €215,000
• VIN: WP0ZZZ99ZPS291047
• Make / Model: Porsche / 911 GT3
• Horsepower: 510 HP
• Mileage: 4,200 km
• Transmission: 7-Speed PDK
• Status: In Stock (1 Unit)
[ ADD TO CART (€215,000) ]
[ ♥ ADD TO WISHLIST (B2) ]

RECOMMENDED VEHICLES (B3): [ 2021 McLaren 720S ] | [ 2022 Lamborghini Huracán ]
```

6.3 Wishlist Wireframe (B2)

```
MY WISHLIST   (WishlistItem: UserId + CarVin)
```

Image	Vehicle	VIN	Price	Availability	Added	Action
[img]	2023 Porsche 911 GT3	WP0ZZZ99ZPS291047	€215,000	In stock	2026-10-30	[Add to cart] [Remove]
[img]	2022 Ferrari F8 Tributo	ZFF92LLA9N0267841	€248,000	Sold	2026-11-02	[Remove]

6.4 Site Reviews Wireframe (B7)

```
WHAT OUR CLIENTS SAY ABOUT RF PRESTIGE                    Average: 4.7 / 5

Andrei M. ★★★★★
"Smooth purchase, the car matched its description."
Ioana P. ★★★★☆
"Great selection, delivery took two days longer."
WRITE A REVIEW (login required)
Rating: [ ★ ★ ★ ★ ☆ ] (1-5)
Comment: [ ..................... ]
[ SUBMIT REVIEW ]
```

7. User Interface Wireframes — Cart, Checkout, Account & Administration

The visual layout mockups for Cart session, Checkout flow, Account area and Administration panel demonstrate administrative workflows and client transactional UX.

7.1 Shopping Cart & Checkout Wireframes

```
YOUR SHOPPING CART (A3)

1. 2023 Porsche 911 GT3 — €215,000
   VIN: WP0ZZZ99ZPS291047  [Remove]

Delivery Method:
(o) Enclosed Trailer (+€490)
( ) Showroom Pickup (€0)
Total Amount: €215,490
[ PROCEED TO CHECKOUT > ]
```

```
CHECKOUT FLOW (A5)

Step 1: Contact & Delivery Address
Recipient: John Doe (from account)
Saved address: [ Str. Aviatorilor 12, Bucharest v ] or [ + New ]
Street: [ Str. Aviatorilor 12 ]
City: [ Bucharest ]  County: [ București ]
Postal Code: [ 011861 ]  Country: [ Romania ]
[x] Save this address to my account
[x] I accept purchase terms.
[ PROCEED TO STRIPE PAYMENT ]
```

7.2 Account Area Wireframe (Order History & Addresses)

```
MY ACCOUNT | Orders (A8) | Addresses | Wishlist
```

MY ORDERS

Order #	Vehicle	VIN	Total	Delivery	Status	Invoice
1042	2023 Porsche 911 GT3	WP0ZZZ99ZPS291047	€215,490	Trailer	Paid	[PDF]

MY ADDRESSES [+ Add Address]

Street	City	County	Postal Code	Country	Default	Action
Str. Aviatorilor 12	Bucharest	București	011861	Romania	[x]	[Edit] [Delete]
Bd. Eroilor 5	Cluj-Napoca	Cluj	400129	Romania	[ ]	[Edit] [Delete]

7.3 Administration Panel — Inventory

```
RF PRESTIGE BACK-OFFICE ADMIN (A9)                    User: Admin (Roberto) | [Logout]

NAV MENU
• Inventory (Cars)
• Orders & Sales
• Site Reviews
• Metrics
```

VEHICLE INVENTORY MANAGEMENT [+ Add New Car]

VIN (PK)	Make / Model	Price	Stock	Action
WP0ZZZ99ZPS291047	2023 Porsche 911 GT3	€215,000	1 (Avail)	[Edit] [Delete]
ZFF92LLA9N0267841	2022 Ferrari F8	€248,000	0 (Sold)	[Edit] [Delete]

Add / Edit form: VIN (17 chars, unique), Make, Model, Price, Horsepower, Mileage, Stock, Images

7.4 Administration Panel — Orders & Site Reviews

```
ADMIN | ORDERS & SALES
```

Order #	Customer	Ship to (City, Postal Code)	Vehicle VIN	Total	Status	Action
1042	john@mail.ro	Bucharest, 011861	WP0ZZZ99ZPS291047	€215,490	Paid	[Update Status]
1043	ana@mail.ro	Cluj-Napoca, 400129	ZFF92LLA9N0267841	€248,000	Pending	[Update Status]

```
ADMIN | SITE REVIEWS (MODERATION)
```

User	Rating	Comment	Date	Action
Andrei M.	5	Smooth purchase, the car matched its description.	2026-11-01	[Delete]
Ioana P.	4	Great selection, delivery took two days longer.	2026-11-03	[Delete]

8. Quality Assurance & Risk Mitigation Plan

Risk Identified	Severity	Primary Cause	Engineered Mitigation Strategy
Local Webhook Drop	High	Stripe webhook fails to reach localhost development environment.	Mandatory Stripe CLI tunnel initialization during startup script; unit test simulating raw payload verification.
Double Stock Allocation	Critical	Concurrent buyers submit payments for same vehicle simultaneously.	EF Core optimistic locking (`[Timestamp]` column) + explicit database transaction isolation.
Invalid or Duplicate VIN	Medium	VIN is the primary key; a mistyped or duplicate VIN would break order and wishlist links.	Primary key uniqueness in SQLite; admin form validates 17 characters (no I, O, Q); unit test for VIN validation.
Shared Address Corruption	Medium	Addresses are shared between users (N:M) and referenced by past orders; an in-place edit would change other users' data and historical shipping details.	Address rows treated as immutable (edit = create/reuse + relink); delete removes only the `UserAddress` link; unit tests for deduplication, relinking and default-address uniqueness.
Scope & Time Slip	Medium	Feature creep blocking delivery of core requirements.	Enforce zero work on Bonus features B1–B7 until xUnit test suite passes 100% for modules A1–A12.

9. Working Prototype Verification (P2 Scope)

As mandated for Presentation 2 (Week 7), the working software prototype demonstrates the following live system components:

- **Database Seed Verification:** SQLite database seeded with 30+ pre-owned luxury vehicles (unique VINs) across all 4 categories.
- **Live Catalog Browsing:** Functional catalog rendering real vehicle specs, pricing, and images directly from the EF Core database layer.
- **Authentication Flow:** User login, registration, and session initialization executing with BCrypt identity security.

10. Execution Timeline & Implementation Roadmap

The 12-week execution strategy ensures balanced workload distribution across both team members.

Week / Milestone	Focus Area & Tasks	Deliverables / Validation Points	Lead
Week 4	Repository setup, Docker Compose environment, EF Core baseline entities (VIN-keyed Car, Address, UserAddress), seed script (30+ vehicles).	Working Docker build, seeded database with luxury vehicles catalog.	Both
Week 5	A4 User Accounts (Identity), A1 Catalog display views, A2 Basic Search query builder.	User login/registration functional, searchable car grid.	Roberto / Alexandru
Week 6	A3 Cart implementation, UI wireframes, system specification completion.	Session cart functional, completed specification document.	Roberto / Alexandru
Week 7	Presentation 2 (P2): Specification defense and functional prototype demonstration.	Live demo of catalog browsing, search, and user authentication.	Both
Week 8	A5 Checkout workflow implementation (address capture and linking), A11 Responsive UI styling refinement.	Multi-step checkout functional, mobile responsive layout verified.	Roberto / Alexandru
Week 9	A6 Stripe payment integration, Webhook receiver, A7 Stock locking logic.	End-to-end sandbox payment flow working with automated stock update.	Roberto
Week 10	A9 Custom Admin Panel, A8 Customer Order History dashboard, B1 Filtered search.	Back-office dashboard online, order history accessible to users.	Alexandru
Week 11	A10 Security hardening, A12 xUnit automated test writing, README, video recording.	100% test passing, security audit completed, final README.	Roberto / Alexandru
Week 12	Presentation 3 (P3): Final live demonstration, code defense, test suite execution.	Fully functional deployed application and project evaluation.	Both

11. Specification Sign-Off & Freezing Notice

This specification is officially frozen as of Presentation 2 (Week 7). What is approved in this document forms the definitive requirement baseline against which Presentation 3 (P3) will be evaluated.

Cătălinoiu Luis Roberto
Backend & Payments Lead

Țicleanu Ștefan Alexandru
Frontend & Administration Lead